🎓 Titre RNCP 37873 — CDA Niveau 6

MyFamilyAppli

Application collaborative d’organisation familiale

Soutenance de projet — Concepteur Développeur d'Applications

STACK TECHNIQUE
React / Node.js Express / MySQL / Docker

PRODUCTION CLOUD
VPS OVH (Debian 12)

Thérèse NGUYEN

27 Août 2026

VISIPLUS academy

01. Introduction

02. Conception

03. Base de Données

04. Développement Applicatif

05. Sécurité Applicative

06. Qualité & Tests

07. Déploiement & DevOps

08. Conclusion & Bilan

01. Introduction

Présentation générale, contexte et périmètre des compétences couvertes

Périmètre RNCP 37873

Bloc 1 — Développer une appli sécurisée

- Docker, Node.js, React, MySQL
- Interfaces UI Bootstrap / React
- Composants métier Back-end
- Gestion de projet Agile & Gantt

Bloc 2 — Concevoir une architecture multicouche

- Personas, User Stories, Maquettes
- Diagrammes UML
- MCD, MLD, MPD Merise
- Accès aux données

Bloc 3 — Préparer le déploiement (DevOps)

- Plan de tests Jest & Vitest
- Conteneurisation Docker Compose
- Reverse Proxy Nginx
- CI/CD GitHub Actions sur VPS

Présentation du projet MyFamilyAppli

Projet personnel

Réalisé dans le cadre du titre CDA

Mise en situation professionnelle simulée

Contexte & Problématique

Charge Mentale Familiale

Gestion du foyer = rendez-vous médicaux
fournitures scolaires,
courses, travaux,
évènements associatifs et familiaux …

⇒ charge mentale pour des parents actifs

Multiplication des outils :

- Informations dispersées
- Manque visibilité
- Diminution de l'efficacité de l'organisation familiale
- Risque de doublons ou d'oubli.

Objectif de MyFamilyAppli

Proposer une application unique, partagée avec les membres d'une même famille et centralisant :

- Calendrier partagé
- To-do list
- Liste de courses

Caractéristiques :

- sobre
- intuitif
- adapté à l'usage Mobile

Solution & Stack technique

Fonctionnalités de MyFamilyAppli

- **Inscription :** Création de l'espace Famille
- **Connexion :** Accès sécurisé à l'application MyFamilyAppli
- **Dashboard :** Affichage des indicateurs clés
- **Calendrier Familial :** Vision unifiée des rendez-vous et évènements du foyer
- **To-do List :** Tâches partagées avec niveaux d'urgence, deadlines et statuts
- **Liste de courses :** Gestion dynamique des articles (à acheter/en stock)

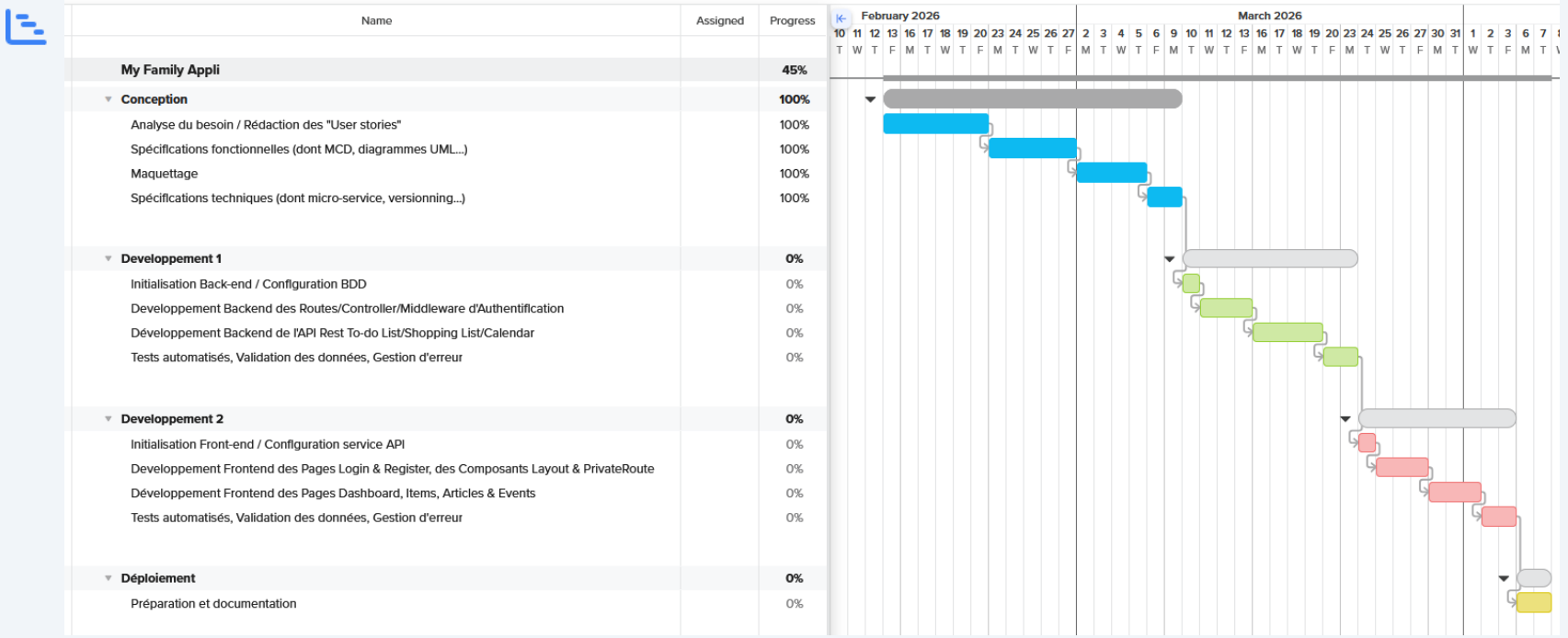
Stack technique

- **Architecture multicouche découplée :**
  - Front-end REACT
  - Back-end Node.js
  - Base de données MySQL
- **API REST**
- **Application sécurisée**
- **Déploiement** Docker / GithHub Actions
- **Mise en production** VPS Cloud Linux

Méthodologie & Planning

Outillage du Projet

Planification (TeamGantt)
Maquettes (Miro)
UML (Visual Paradigm v18.0)
Modèles de données (JMerise v6.2)
IDE (Visual Studio Code)
Versionning (GitHub)
Conteneurisation (Docker)
CI/CD (GitHub Actions)



Name	Assigned	Progress
My Family Appli		**45%**
Conception		**100%**
Analyse du besoin / Rédaction des "User stories"		100%
Spécifications fonctionnelles (dont MCD, diagrammes UML...)		100%
Maquettage		100%
Spécifications techniques (dont micro-service, versionning...)		100%
Developpement 1		**0%**
Initialisation Back-end / Configuration BDD		0%
Developpement Backend des Routes/Controller/Middleware d'Authentification		0%
Développement Backend de l'API Rest To-do List/Shopping List/Calendar		0%
Tests automatisés, Validation des données, Gestion d'erreur		0%
Developpement 2		**0%**
Initialisation Front-end / Configuration service API		0%
Developpement Frontend des Pages Login & Register, des Composants Layout & PrivateRoute		0%
Développement Frontend des Pages Dashboard, Items, Articles & Events		0%
Tests automatisés, Validation des données, Gestion d'erreur		0%
Déploiement		**0%**
Préparation et documentation		0%

[Planning Gantt Indicatif du Projet]

Approche Agile itérative : Adapté au contexte de développement individuel

+ Sprints successifs pour livrer rapidement un MVP fonctionnel

Démarche projet mené de bout en bout :

Analyse du besoin → Conception → Développement →Tests → Déploiement → Mise en production

02. Conception

Analyse du besoin, Spécifications fonctionnelles, maquettage UX/UI et modélisation UML

Analyse des Personas

Cédric (50 ans)

Cadre supérieur, peu de temps.

Veut réserver l'agenda pour des intervenants à la maison sans polluer son calendrier pro.

Karine (45 ans)

Profession libérale à domicile.

Apprécie les choses très bien organisées pour orchestrer la maison.

Lucas (Lycéen)

Joue du piano, aime la musique.

Besoin de transmettre ses demandes de fournitures et réserver des déplacements pour ses sorties.

Jade (Collégienne)

Très connectée (sans forfait data dédié).

Utilise le Wi-Fi maison pour sa wish-list et besoin d'accompagnement aux activités sportives.

User stories

En tant que Parent, **je veux** pouvoir planifier les évènements liés aux adolescents **afin qu**'ils aient la visibilité de leurs activités/rdv.

En tant que Parents, **je veux** pouvoir disposer de la liste des courses **afin d'**aller au supermarché avec une liste complète des besoins de chacun.

En tant qu'Adolescent, **je veux pouvoir** assigner à mas Parents une tâche à réaliser (ex : inscription à la prochaine sortie scolaire) **afin d'**organiser la vie quotidienne et ne rien oublier.

User Stories & Backlog

User stories majeures

- **US-01 :** En tant que nouvel utilisateur, je veux pouvoir créer/rejoindre mon espace famille sécurisé
- **US-02 :** En tant que membre, je veux pouvoir me connecter à mon espace famille
- **US-03 :** En tant que membre, je veux pouvoir ajouter un évènement au calendrier de la famille
- **US-04 :** En tant que membre, je veux pouvoir ajouter une tâche à la To-do List
- **US-05 :** En tant que membre, je veux pouvoir supprimer une tâche de la To-do List une fois celle-ci terminée
- **US-06 :** En tant que membre, je veux pouvoir ajouter un article à la liste de courses
- **US-07 :** En tant que membre, je veux pouvoir supprimer un article de la liste de courses une fois celui-ci acheté

Arbitrages du Backlog

Features reportées (contraintes de temps) pour garantir la stabilité du cœur fonctionnel :

- Gestion différenciée des rôles Parents / Adolescents
- Modification directe d’évènements existants
- Système de notification-push
- Assignation de chaque évènement et/ou tâche à un membre de la famille (participants/pilote)

Diagramme UML Cas d’Utilisation

Acteurs principaux :

Membre Familial (Parent / Teen)

Actions :

S’inscrire

Se connecter

Consulter le Dashboard

Actions Calendrier :

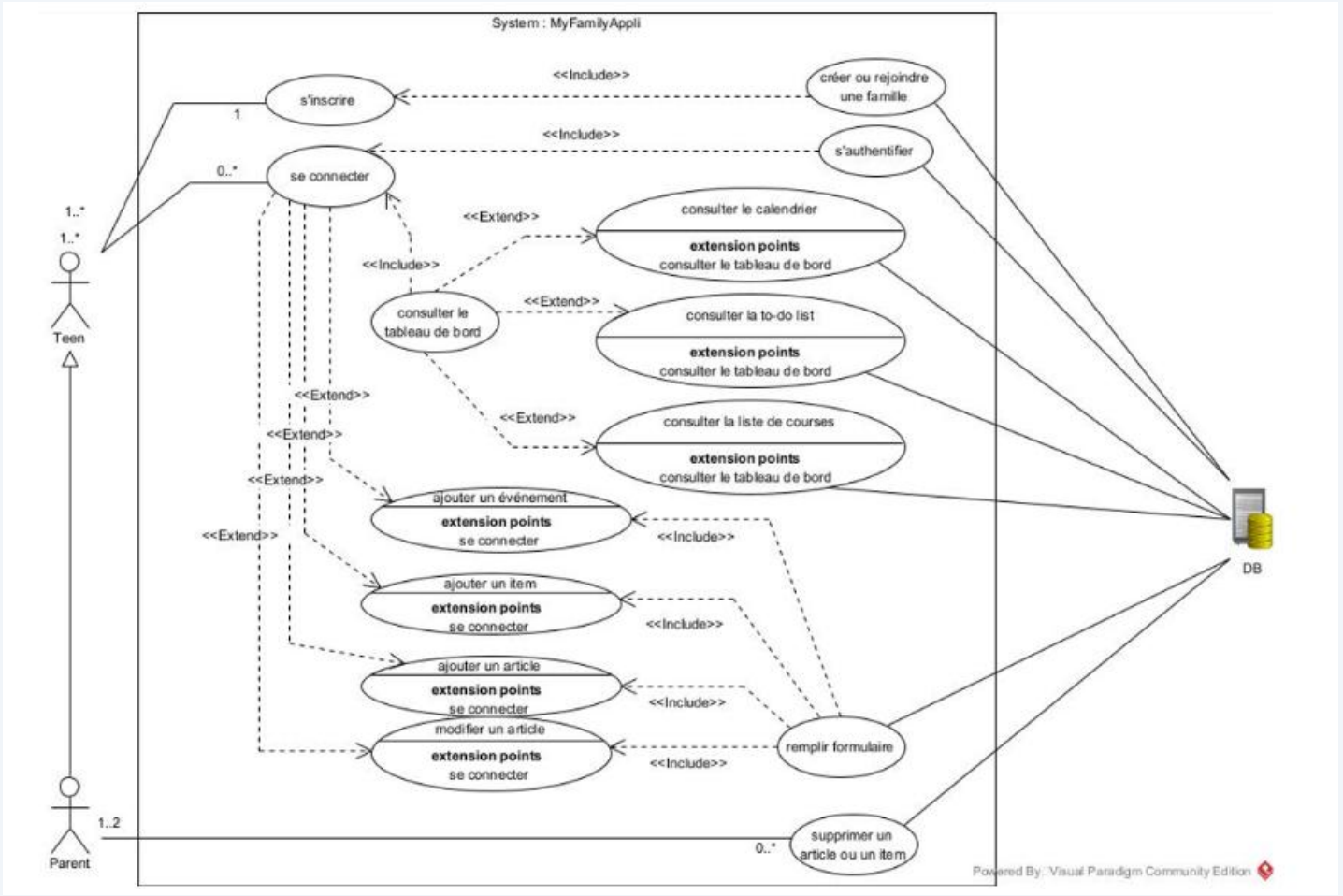
Consulter, Ajouter un évènement

Actions To-do List :

Consulter, Ajouter, Supprimer un item

Actions Liste de courses :

Consulter, Ajouter, Modifier, Supprimer un article

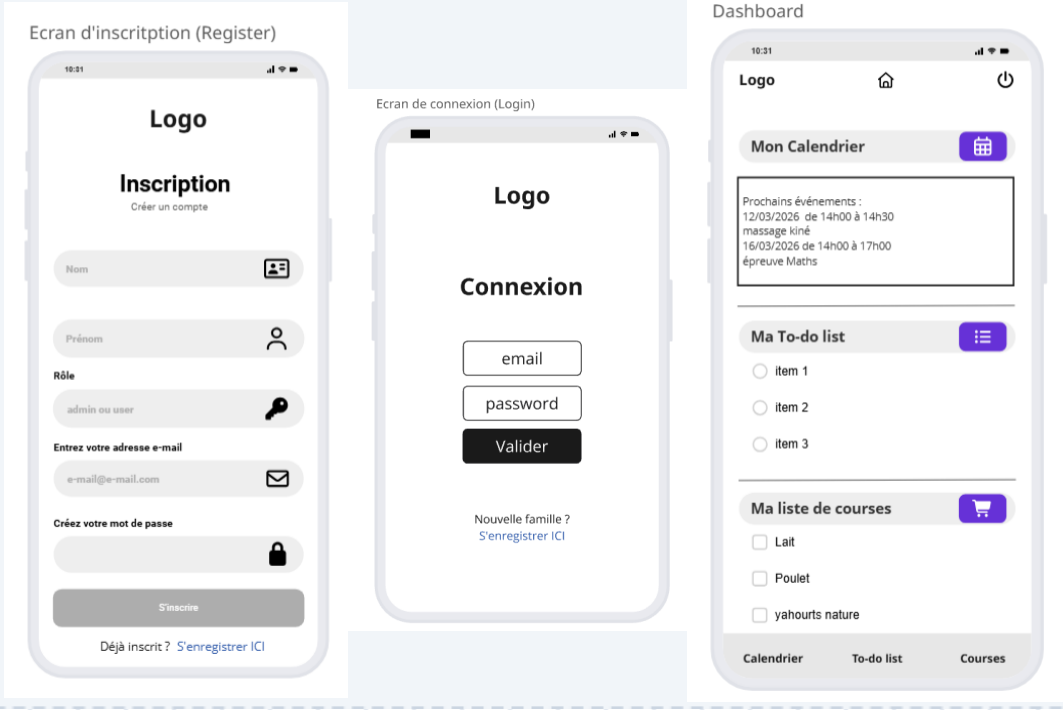


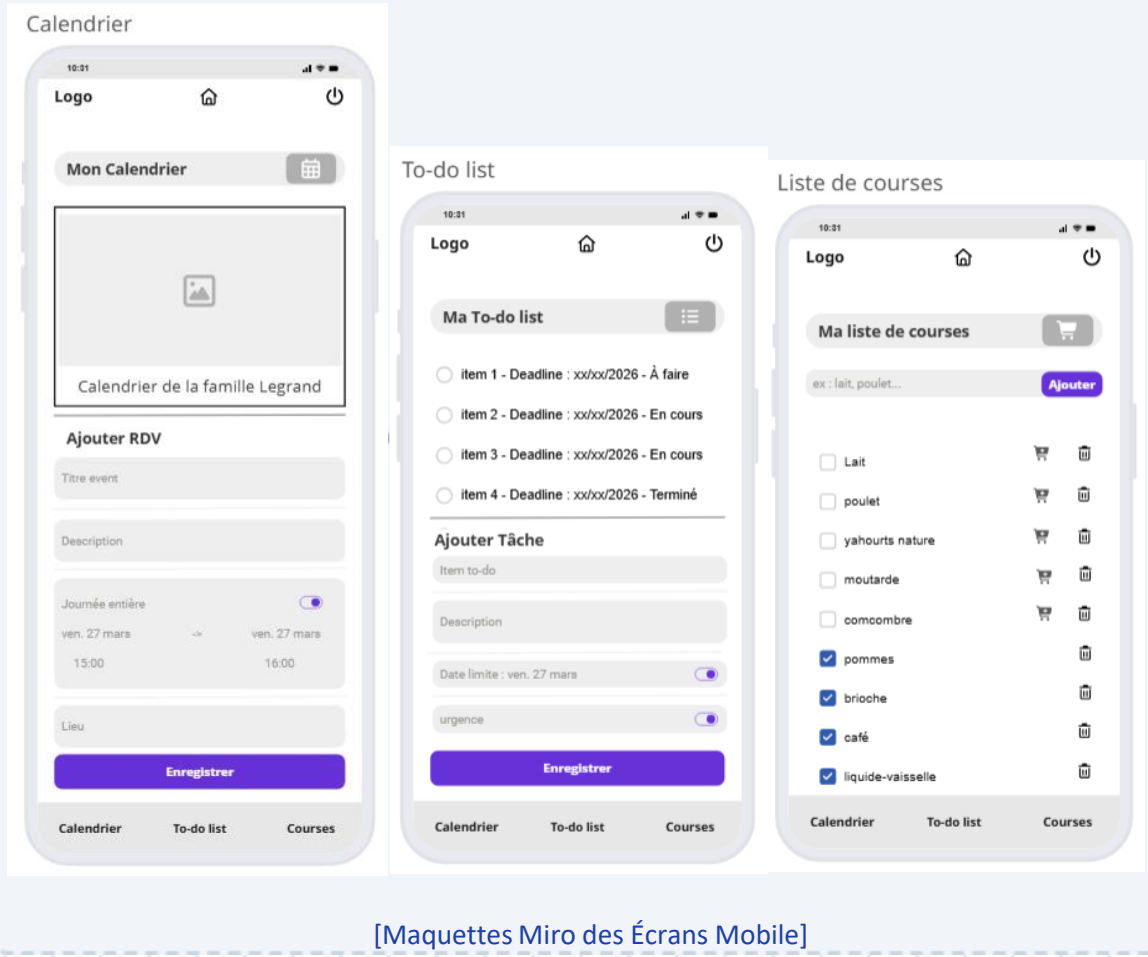
[Diagramme UML de Cas d'Utilisation]

Conception UX & Maquettes

Principes UI Mobile-First

- Maquettage de 6 écrans sous Miro.
- Barre de navigation fixe en bas d'écran.
- Boutons et zones de clics larges adaptés aux pouces.
- Feedback visuel immédiat après action.



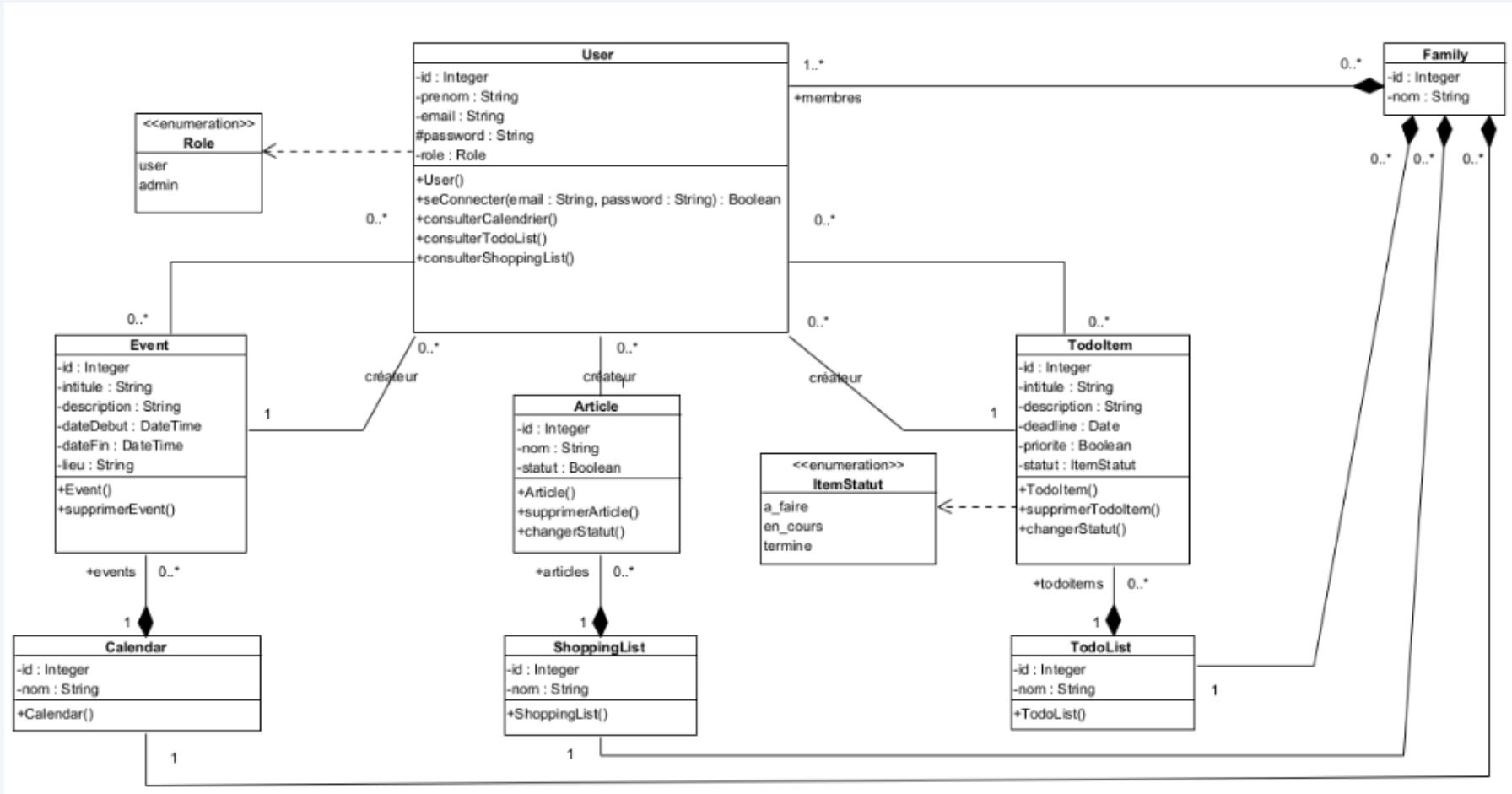


[Maquettes Miro des Écrans Mobile]

Diagramme de Classes

Structure des classes métier :

- **Family :** Entité pivot regroupant plusieurs membres | **User :** Membre avec rôle (user/admin) et mot de passe chiffré
- **TodoList & TodoItem :** Tâches | **ShoppingList & Article :** Ingrédients et fournitures | **Calendar & Event :** Rdv et événements

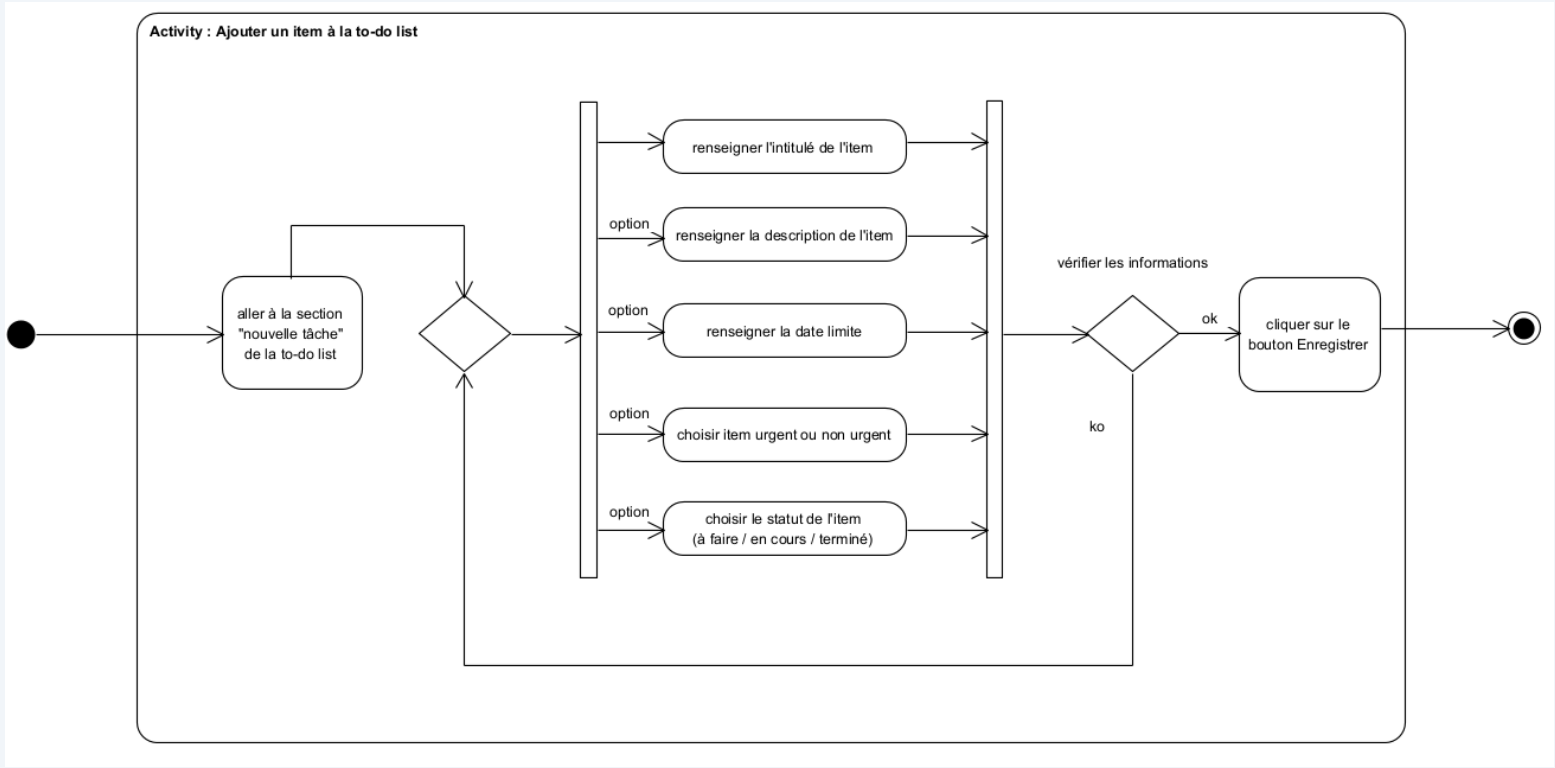


[Diagramme UML de Classes]

Diagramme d'Activité

Scénario : Ajouter une Tâche

1. Sur la section « Nouvelle Tâche » de la to-do list :
2. Saisie parallèle des champs : Intitulé (obligatoire), Description, Date limite, Priorité, Statut
3. Vérification et contrôle des données saisies
4. Enregistrement en BDD



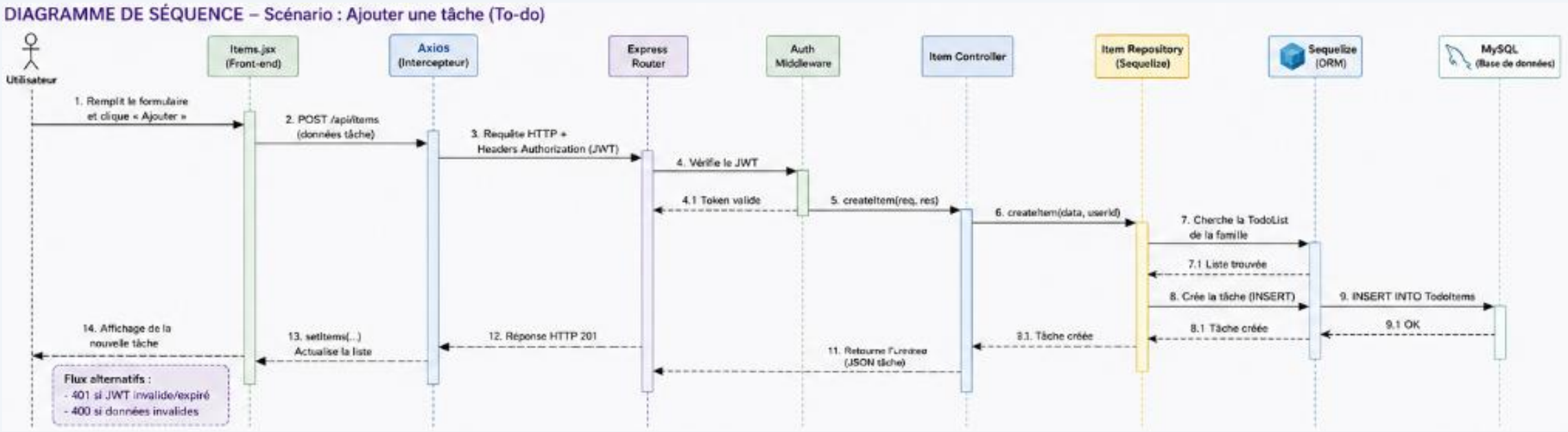
[Diagramme UML d'Activité "Ajouter un Item"]

Diagramme de Séquence

Scénario : Ajouter une Tâche

Traversée des couches :

1. React Client -> Axios (Header Authorization JWT)
2. Express Router -> AuthMiddleware (Vérification Token)
3. ItemController -> ItemRepository
4. Sequelize ORM -> MySQL (INSERT SQL)
5. Retour HTTP 201 Created & mise à jour du State React



[Diagramme UML de Séquence "Ajouter Tâche"]

Critères d'Acceptation (extrait)

User Story	Critères d'Acceptation (Gherkin BDD)	Réf Test
Inscrire Utilisateur	**Étant donné** identifiants valides et password >= 8 caractères, **Quand** soumission, **Alors** compte créé et token JWT retourné (HTTP 201).	AUTH-1
Se Connecter	**Étant donné** mot de passe invalide, **Quand** soumission, **Alors** rejet HTTP 401 Unauthorized.	AUTH-5
Créer Tâche	**Étant donné** libellé vide, **Quand** soumission, **Alors** rejet HTTP 400 Bad Request.	TO-DO-4
Supprimer Item	**Étant donné** tâche existante, **Quand** clic Supprimer, **Alors** retrait BDD et retour HTTP 200 OK.	TO-DO-5

BDD (Behaviour Driven Development) :

Étant donné *[contexte],* ***Quand*** *[action utilisateur],* ***Alors*** *[résultat attendu]*

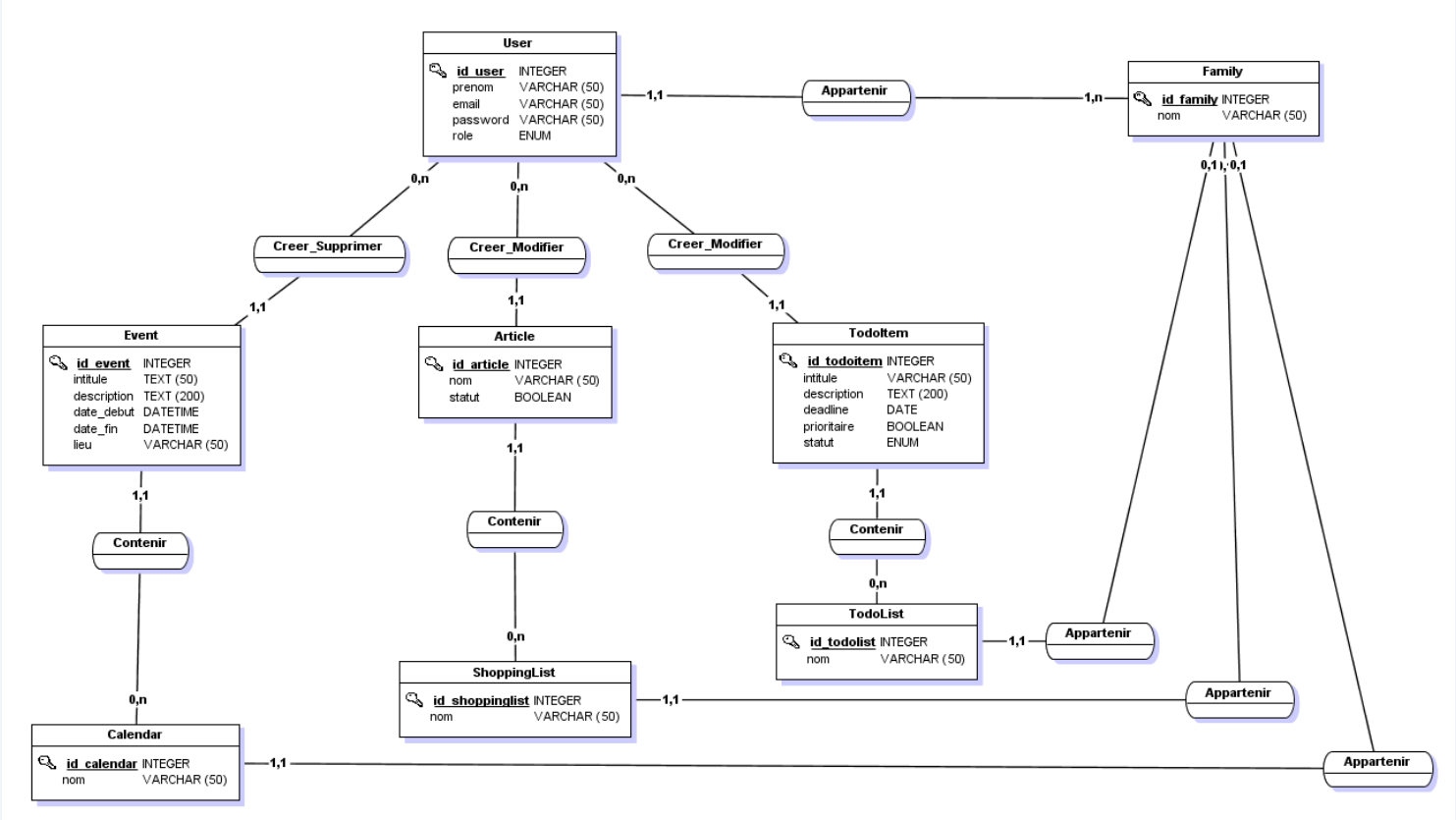
03. Base de Données

Modélisation relationnelle Merise et implémentation MySQL

Modèles MCD et MLD

Méthode Merise :

- **MCD :** Identification des 8 entités et des cardinalités (ex: Family 1,n <---> 1,1 User).
- **MLD :** Traduction en tables avec clés primaires (PK) et clés étrangères (FK)
- **Intégrité référentielle :** Suppressions en cascade (ON DELETE CASCADE)



[Modèle Conceptuel des Données (MCD)]

Modèle MPD

Choix SGBDR : MySQL 8.0 (InnoDB) / Garantie des transactions ACID et du respect strict des contraintes d'intégrité référentielle

Accès aux données via ORM Sequelize / Représentation des tables sous forme de modèles JS, limite la manipulation directe SQL

```sql
-- ----------------------------
-- Table: Family
-- ----------------------------
CREATE TABLE Families (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  createdAt DATETIME NOT NULL,
  updatedAt DATETIME NOT NULL
);

-- ----------------------------
-- Table: User
-- ----------------------------
CREATE TABLE Users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  prenom VARCHAR(50) NOT NULL,
  email VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(50) NOT NULL,
  role ENUM('user', 'admin') NOT NULL DEFAULT 'user',
  createdAt DATETIME NOT NULL,
  updatedAt DATETIME NOT NULL,
  familyId INT NOT NULL,
  CONSTRAINT FOREIGN KEY (familyId) REFERENCES Families (id)
  ON DELETE CASCADE ON UPDATE CASCADE
);
```

```sql
-- ----------------------------
-- Table: TodoList
-- ----------------------------
CREATE TABLE TodoLists (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  createdAt DATETIME NOT NULL,
  updatedAt DATETIME NOT NULL,
  familyId INT NOT NULL,
  CONSTRAINT FOREIGN KEY (familyId) REFERENCES Families (id)
  ON DELETE CASCADE ON UPDATE CASCADE
);

-- ----------------------------
-- Table: TodoItem
-- ----------------------------
CREATE TABLE TodoItems (
  id INT AUTO_INCREMENT PRIMARY KEY,
  intitule VARCHAR(50) NOT NULL,
  description VARCHAR(200) NULL,
  deadline DATE NULL,
  priorite TINYINT(1) NOT NULL DEFAULT 0,
  statut ENUM('a_faire', 'en_cours', 'termine') NOT NULL DEFAULT 'a_faire',
  createdAt DATETIME NOT NULL,
  updatedAt DATETIME NOT NULL,
  userId INT NULL,
  todolistId INT NULL,
  CONSTRAINT FOREIGN KEY (userId) REFERENCES Users (id)
  ON DELETE SET NULL ON UPDATE CASCADE
  CONSTRAINT FOREIGN KEY (todolistId) REFERENCES TodoLists (id)
  ON DELETE CASCADE ON UPDATE CASCADE
);
```

[Modèle Physique des Données (MPD) - extrait]

Comparatif SQL vs NoSQL

Critère de choix	Relationnel SQL (MySQL) — RETENU	NoSQL Orienté Document (ex: MongoDB)
Structure des données	Fortement structurée & relations claires	Schéma souple / Documents imbriqués
Intégrité Référentielle	Clés étrangères & Transactions ACID natives	Gestion applicative complexe des relations
Cohérence des données	Garantie sans duplication	Risque d'incohérence si duplication
Verdict Projet	**Parfaitement adapté au contexte familial**	Inutilement complexe pour notre besoin

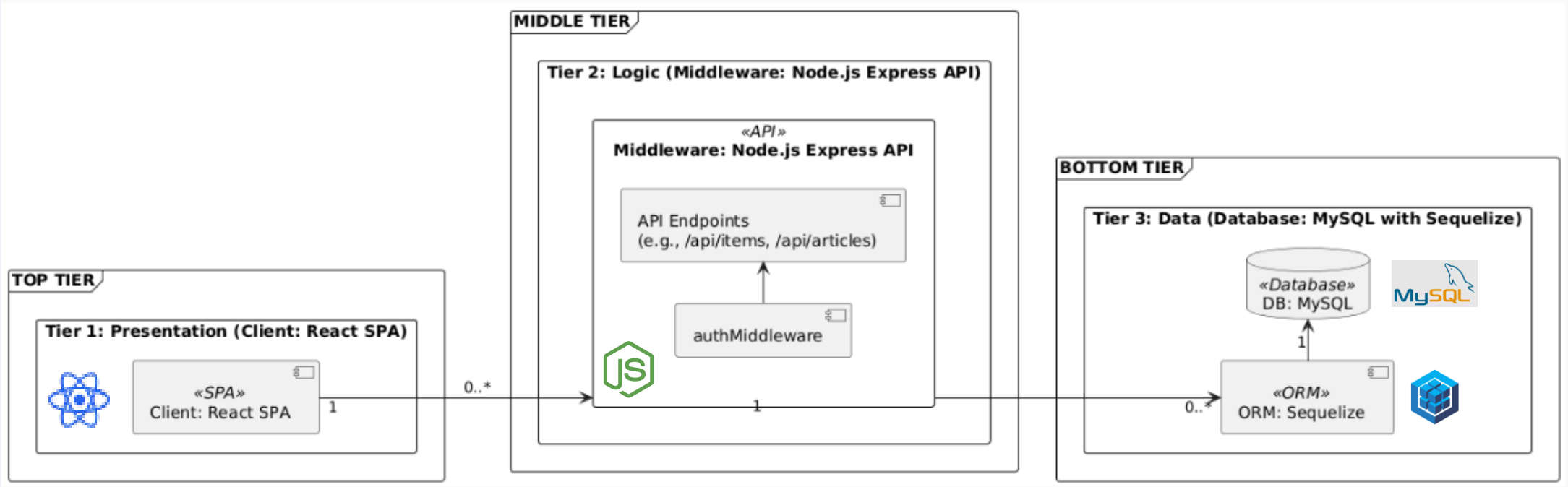
04. Développement Applicatif

Architecture 3-Tiers, développements Back-end Node.js, Front-end

Architecture multicouche

Découplage complet des responsabilités :

- **Tier 1 — Présentation :** Single Page Application (SPA) avec React.
- **Tier 2 — Métier :** API REST Node.js Express (Asynchrone).
- **Tier 3 — Données :** MySQL 8.0 via l'ORM Sequelize.



[Schéma d'Architecture 3-Tiers]

Architecture Back-end

Routes

Endpoints RESTful (/api/items, /api/events, etc.) gérant le routage HTTP

Middlewares

Validation Express-Validator & contrôle du JWT (authMiddleware)

Controllers

Traitement HTTP, gestion des requêtes, réponses et codes d'erreur.

Repositories

Pattern Repository : Isolation totale des requêtes BDD Sequelize.



```
mfa-backend/
├── server.js
├── app.js
├── config/
│   └── database.js
├── models/
│   ├── User.js
│   ├── Family.js
│   ├── Calendar.js
│   ├── Event.js
│   ├── TodoList.js
│   ├── TodoItem.js
│   ├── index.js
│   └── ...
├── routes/
│   ├── authRoutes.js
│   ├── userRoutes.js
│   ├── eventRoutes.js
│   ├── itemRoutes.js
│   └── articleRoutes.js
├── middleware/
│   ├── authMiddleware.js
│   └── validateMiddleware.js
├── validators/
│   └── authValidator.js
├── controllers/
│   ├── authController.js
│   ├── eventController.js
│   ├── itemController.js
│   └── articleController.js
├── repositories/
│   ├── authRepository.js
│   ├── eventRepository.js
│   ├── itemRepository.js
│   └── articleRepository.js
├── tests/
│   ├── auth.test.js
│   ├── calendar.test.js
│   └── item.test.js
├── Dockerfile
├── .env
└── package.json
```

[Arborescence du Projet Back-end]

Front-end React & Vite

Stack & Gestion d'État

- **Vite :** Build & serveur de dev ultra-rapide.
- **Hooks React :** useState (état local des listes) et useEffect (chargement API).
- **Bootstrap & Toastify :** Responsive & notifications d'erreurs/succès.

Routage Sécurisé

avec composant PrivateRoute.jsx :

```jsx
const PrivateRoute = () => {
  const token = localStorage.getItem('token');

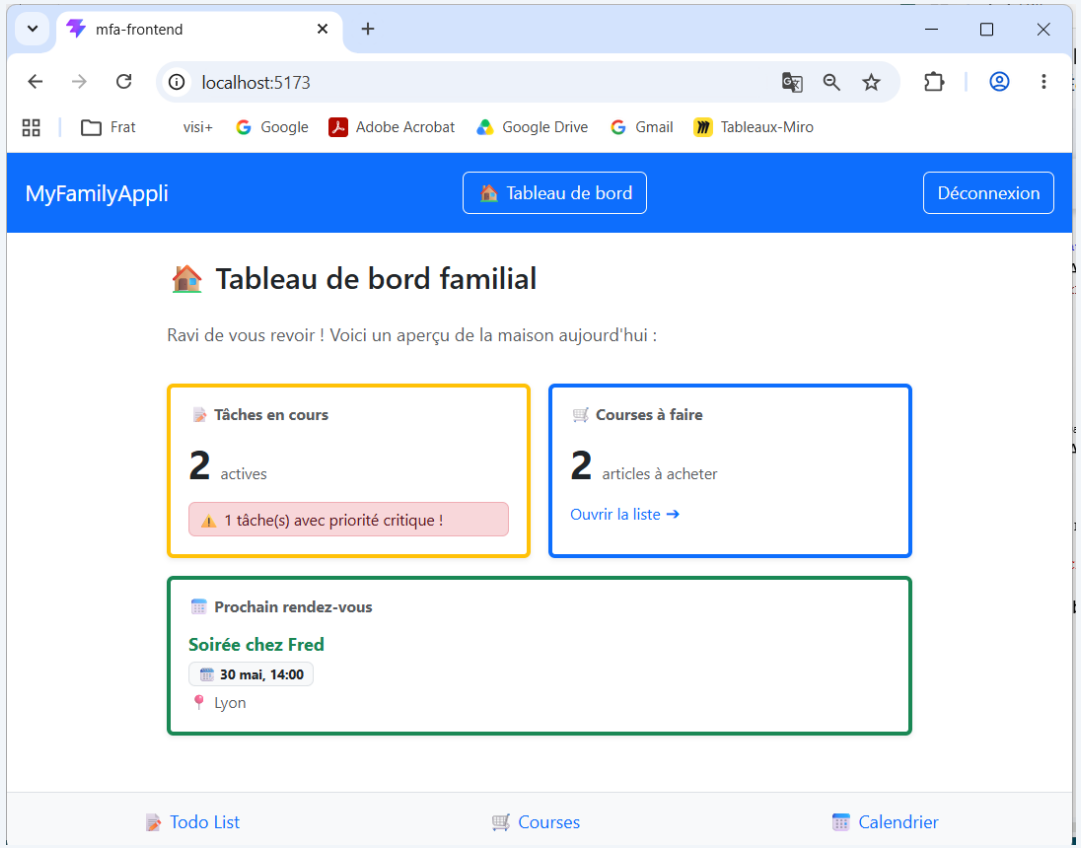
  // Si token présent, on affiche l'Outlet (le contenu), sinon on renvoie au Login
  return token ? <Outlet /> : <Navigate to="/login" />;
};
```



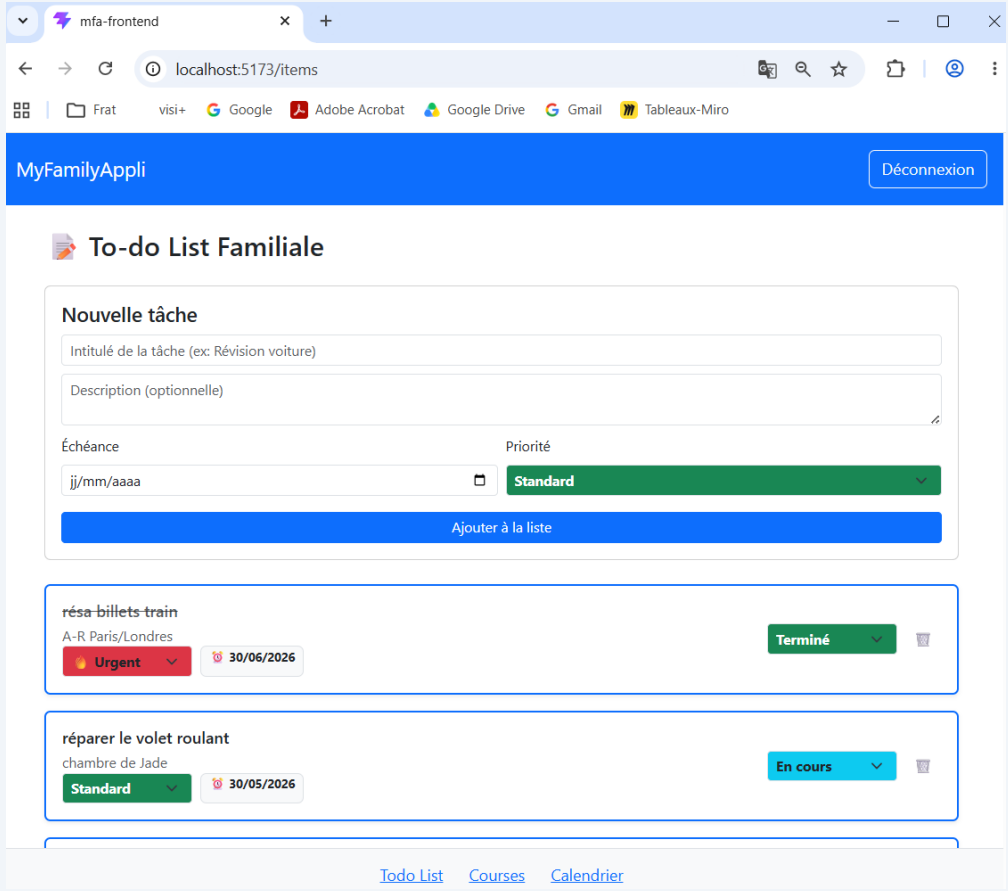
```
mfa-frontend/
├── src
│   ├── __tests__
│   │   ├── Dashboard.test.jsx
│   │   └── Login.test.jsx
│   ├── components/
│   │   ├── Layout.jsx
│   │   └── PrivateRoute.jsx
│   ├── pages/
│   │   ├── Dashboard.jsx
│   │   ├── Articles.jsx
│   │   ├── Events.jsx
│   │   ├── Items.jsx
│   │   ├── Login.jsx
│   │   └── Register.jsx
│   ├── services/
│   │   └── api.js
│   ├── App.jsx
│   └── main.jsx
├── Dockerfile
├── index.html
└── package.json
```

[Arborescence du Projet Back-end]

Interfaces Utilisateur



[Capture Dashboard Familial]



[Captures To-do List & Calendrier]

05. Sécurité Applicative

Protection des données personnelles (RGPD), authentification JWT, bcrypt et filtres anti-injections

Stratégie de Sécurité

Respect des principes RGPD : Minimisation des données collectées (nom, prénom, email, mot de passe chiffré uniquement).

Les 4 Verrous de Sécurité

1. Validation des entrées (Express-Validator).
2. Hachage robuste des mots de passe (Bcrypt).
3. Authentification Stateless par JWT.
4. Isolation des requêtes SQL par famille.

```
1. Login
      ↓
2. Validation des données (Express Validator)
      ↓
3. Vérification utilisateur en base
      ↓
4. Comparaison bcrypt
      ↓
5. Génération JWT
      ↓
6. Envoi au client React
      ↓
7. JWT envoyé dans Authorization: Bearer xxx
      ↓
8. Middleware d'authentification
      ↓
9. Accès à la ressource protégée
```

[Schéma des Verrous Sécurité]

Validation Express & Bcrypt

1. Validation Express-Validator (authValidator.js) :

```
exports.registerValidator = [
  body('email').trim().isEmail().withMessage('Format de l\'email invalide.').normalizeEmail(),
  body('password').isLength({ min: 8 }).withMessage('Le mot de passe doit contenir au moins 8 caractères.'),
  body('prenom').notEmpty().trim().escape().withMessage('Le prénom est obligatoire.'),
  body('nom')
    .if((value, { req }) => !req.body.familyId) // Condition : si familyId est absent
    .notEmpty().trim().escape().withMessage('Le nom est obligatoire pour créer une nouvelle famille'),
  body().custom(body => {
    if (!body.nom && !body.familyId) {
      throw new Error('Vous devez spécifier un nom de famille ou un ID de famille existante.');
    }
    return true;
  }),
  validateRequest
];
```

2. Hachage Bcrypt (authController.js) :

```
// Hachage du mot de passe lors de l'inscription
const hashedPassword = await bcrypt.hash(password, 10);

// Comparaison du mot de passe en clair avec le hash en BDD lors de la connexion
const isPasswordValid = await bcrypt.compare(password, user.password);
```

🔑 Authentification JWT

3. Jeton JSON Web Token (JWT) :

- Signé numériquement avec clé secrète (JWT_SECRET).
- Envoie les claims : userId, role, familyId.
- Durée de validité limitée (24h).

4. Intercepteurs Axios :

Request : Injection automatique de Authorization: Bearer .

Response : Capture globale des erreurs 401/403 et redirection automatique vers /login.

06. Qualité & Tests

Stratégie de tests automatisés, Jest, Supertest et Vitest

Stratégie de Tests

Isolation BDD de Test

- Base dédiée myfamily_test_db sous MySQLWorkBench.
- Réinitialisation forcée avant la suite de tests (sequelize.sync({ force: true })).
- Seeding technique fixe pour garantir l'idempotence.

Frameworks de Tests

- **Back-end :** Jest & Supertest (Intégration API REST).
- **Front-end :** Vitest & React Testing Library (Composants UI).
- **Manuels :** Thunder Client.

Tests Jest

Test API Back-end (Jest/Supertest) :

```
beforeAll(async () => {
  // 1. Réinitialisation complète de la base de test
  await sequelize.sync({ force: true });
  // 2. SEEDING TECHNIQUE : On crée un utilisateur de test pour générer un Token
  await request(app)
    .post('/api/auth/register')
    .send({
      nom: 'Legrand',
      prenom: 'Jean',
      email: 'jean@legrand.com',
      password: 'SuperPassword123!',
      role: 'admin'
    });
  // 3. On connecte cet utilisateur pour récupérer le Token JWT
  const loginRes = await request(app)
    .post('/api/auth/login')
    .send({
      email: 'jean@legrand.com',
      password: 'SuperPassword123!'
    });

  authToken = loginRes.body.token; // Le token est sauvegardé pour toute la suite
});

afterAll(async () => {
  await sequelize.close();
});
```

Tests Jest

Test API Back-end (Jest/Supertest) :

```
describe('API To-do List (Sécurisée)', () => {

  it('Devrait refuser l\'accès à la todo list si le Token est absent', async () => {
    const res = await request(app).get('/api/items');

    expect(res.statusCode).toEqual(401);
    expect(res.body).toHaveProperty('message', 'Accès refusé : Aucun token fourni.');
  });

  it('Devrait autoriser l\'accès et renvoyer la liste (vide au départ) avec un Token valide', async () => {
    const res = await request(app)
      .get('/api/items')
      .set('Authorization', `Bearer ${authToken}`); // Injection du Token dans le Header

    expect(res.statusCode).toEqual(200);
    expect(res.body).toHaveProperty('TodoItems');
    expect(res.body.TodoItems).toEqual([]); // Tableau vide au début
  });
```

Tests Vitest

Test UI Front-end (Vitest/ReactTestingLibrary) :

```
describe('Dashboard', () => {

  beforeEach(() => {
    vi.clearAllMocks();
  });

  /**
   * Mock par défaut
   */
  const mockApi = ({
    items = [],
    articles = [],
    events = [],
  } = {}) => {
```

```
it('devrait afficher le prochain rendez-vous', async () => {

  mockApi({

    events: [
      {
        intitule: 'Dentiste',
        dateDebut: '2099-12-31T10:00:00',
        lieu: 'Lyon'
      }
    ]

  });

  render(
    <BrowserRouter>
      <Dashboard />
    </BrowserRouter>
  );

  expect(
    await screen.findByText(/Dentiste/i)
  ).toBeInTheDocument();

  expect(
    screen.getByText(/Lyon/i)
  ).toBeInTheDocument();

});
```

Bilan des Campagnes

```
PASS tests/auth.test.js
  API Authentification
    √ Devrait créer une nouvelle famille et un utilisateur (157 ms)
    √ Devrait refuser une inscription avec un mot de passe trop court (12 ms)
    √ Devrait refuser une inscription avec un email non valide (14 ms)
  API Connexion
    √ Devrait connecter un utilisateur et renvoyer un JWT (92 ms)
    √ Devrait refuser la connexion avec un mot de passe incorrect (73 ms)
    √ Devrait neutraliser les tentatives d'injection SQL à la connexion (8 ms)

PASS tests/item.test.js
  API To-do List (Sécurisée)
    √ Devrait refuser l'accès à la todo list si le Token est absent (14 ms)
    √ Devrait autoriser l'accès et renvoyer la liste (vide au départ) avec un Token valide (23 ms)
    √ Devrait permettre à l'utilisateur de créer une tâche (37 ms)
    √ Devrait refuser la création d'une tâche sans intitulé (19 ms)
    √ Devrait supprimer une tâche existante (64 ms)

PASS tests/calendar.test.js
  API Calendar
    √ Devrait rejeter un événement dont la date de fin est antérieure à la date de début (15 ms)

Test Suites: 3 passed, 3 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        6.293 s
Ran all test suites.
```

```
RUN  v4.1.7 C:/Users/franc/Desktop/Maman/Reconversion pro 2025/VISIPLUS FORMATION 2025/Dossier Soutenance/MyFamilyA
-frontend
      Coverage enabled with v8

 ✓ src/__tests__/Dashboard.test.jsx > Dashboard > devrait afficher le titre du dashboard 205ms
 ✓ src/__tests__/Dashboard.test.jsx > Dashboard > devrait calculer correctement les indicateurs 14ms
 ✓ src/__tests__/Dashboard.test.jsx > Dashboard > devrait afficher le prochain rendez-vous 47ms
 ✓ src/__tests__/Dashboard.test.jsx > Dashboard > devrait afficher "Aucun événement à venir" 14ms
 ✓ src/__tests__/Dashboard.test.jsx > Dashboard > devrait appeler les trois endpoints 7ms
 ✓ src/__tests__/Login.test.jsx > Composant Login > doit afficher le titre et les champs du formulaire 181ms

 Test Files  2 passed (2)
      Tests  6 passed (6)
   Start at  01:10:46
   Duration  2.28s (transform 192ms, setup 0ms, import 1.00s, tests 475ms, environment 2.39s)
```

Campagne de Test	Outil utilisé	Nombre de tests	Succès	Taux de Réussite
Backend API REST	Jest + Supertest	12	12	**100%**
Frontend UI	Vitest	6	6	**100%**
Tests Manuels	Thunder Client	18	18	**100%**
TOTAL GENERAL	**Multi-outils**	**36**	**36**	**100% PASS**

07. Déploiement & DevOps

Conteneurisation Docker Compose, Reverse Proxy Nginx et pipeline CI/CD GitHub Actions sur VPS Cloud

Conteneurisation Docker

Frontend Nginx

Conteneur distribuant le build statique React. Écoute sur port 80/5173.

Backend Express

Conteneur Node.js API. Écoute sur port interne 5000.

Database MySQL

Instance MySQL 8.0 couplée au volume persistant db_data.

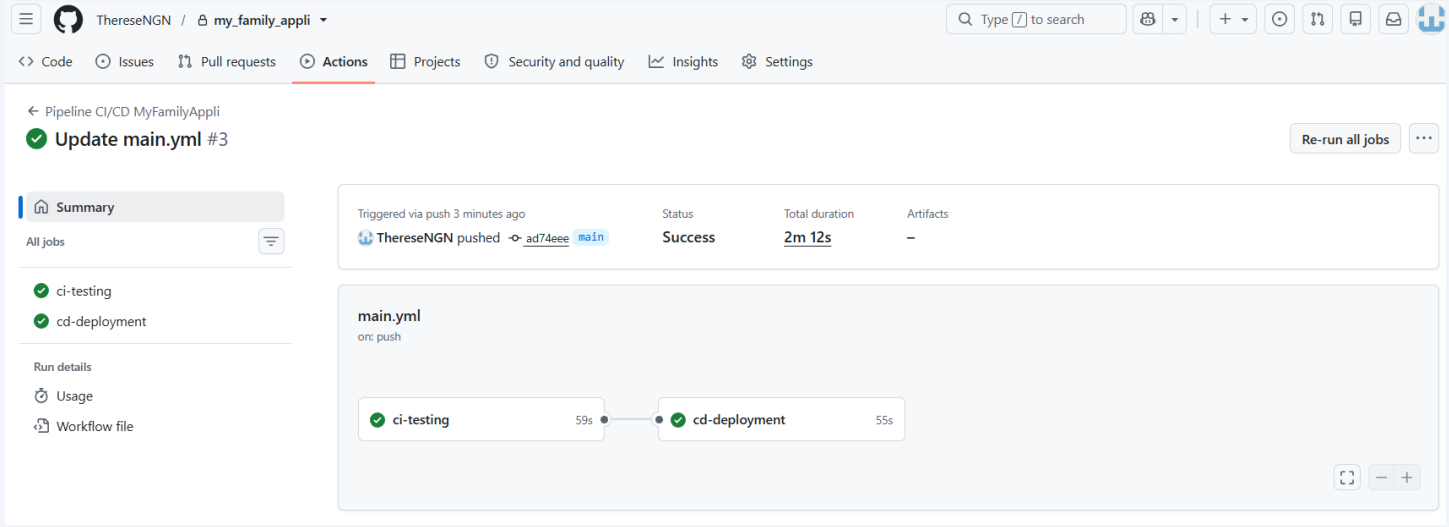
Orchestration Docker Compose & Reverse Proxy Nginx

Nginx intercepte les requêtes /api/ et les redirige de manière transparente vers le service backend interne (http://backend:5000/api/), éliminant nativement les problèmes de CORS.

CI/CD & VPS Cloud OVH

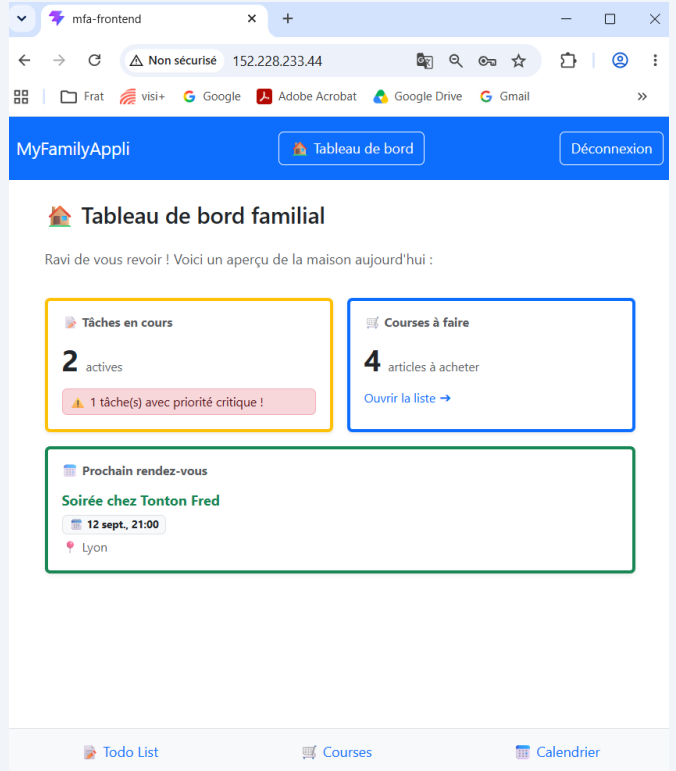
Pipeline GitHub Actions (main.yml) :

- **Phase 1 — CI :** À chaque git push, exécution automatisée des tests Jest sur runner GitHub éphémère.
- **Phase 2 — CD :** Connexion SSH au VPS OVH (Debian 12), git pull et re-build des conteneurs sans interruption.




```
debian@vps-26e085ab:~$ docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS          PORTS                                      NAMES
bd65a662263e   my_family_appli-frontend   "/docker-entrypoint.…"   8 minutes ago    Up 8 minutes    0.0.0.0:80->80/tcp, [::]:80->80/tcp        mfa_frontend_prod
957a57b7e574   my_family_appli-backend    "docker-entrypoint.s…"   8 minutes ago    Up 8 minutes    5000/tcp                                   mfa_backend_prod
d968f561c8c6   mysql:8.0                  "docker-entrypoint.s…"   8 minutes ago    Up 8 minutes    3306/tcp, 33060/tcp                        mfa_db_prod
```

[Workflow GitHub Actions & conteneurisation de l'application]



[Mise en production de MyFamilyAppli]

08. Conclusion & Bilan

Bilan global du projet, enseignements tirés de la reconversion et perspectives d'évolution

Bilan & Perspectives

Résultats & Acquis

- Application full-stack fonctionnelle, testée et déployée.
- Maîtrise du cycle de vie complet d'un projet informatique.
- Mise en pratique des 11 compétences du Référentiel.

Perspectives d'évolution

- Passage du serveur VPS sous HTTPS.
- Gestion des rôles et autorisations.
- Ajout de notifications push.
- Transformation en Progressive Web App



Questions & Réponses

Merci pour votre attention !

Projet MyFamilyAppli — Soutenance Titre RNCP 37873 (Niveau 6)

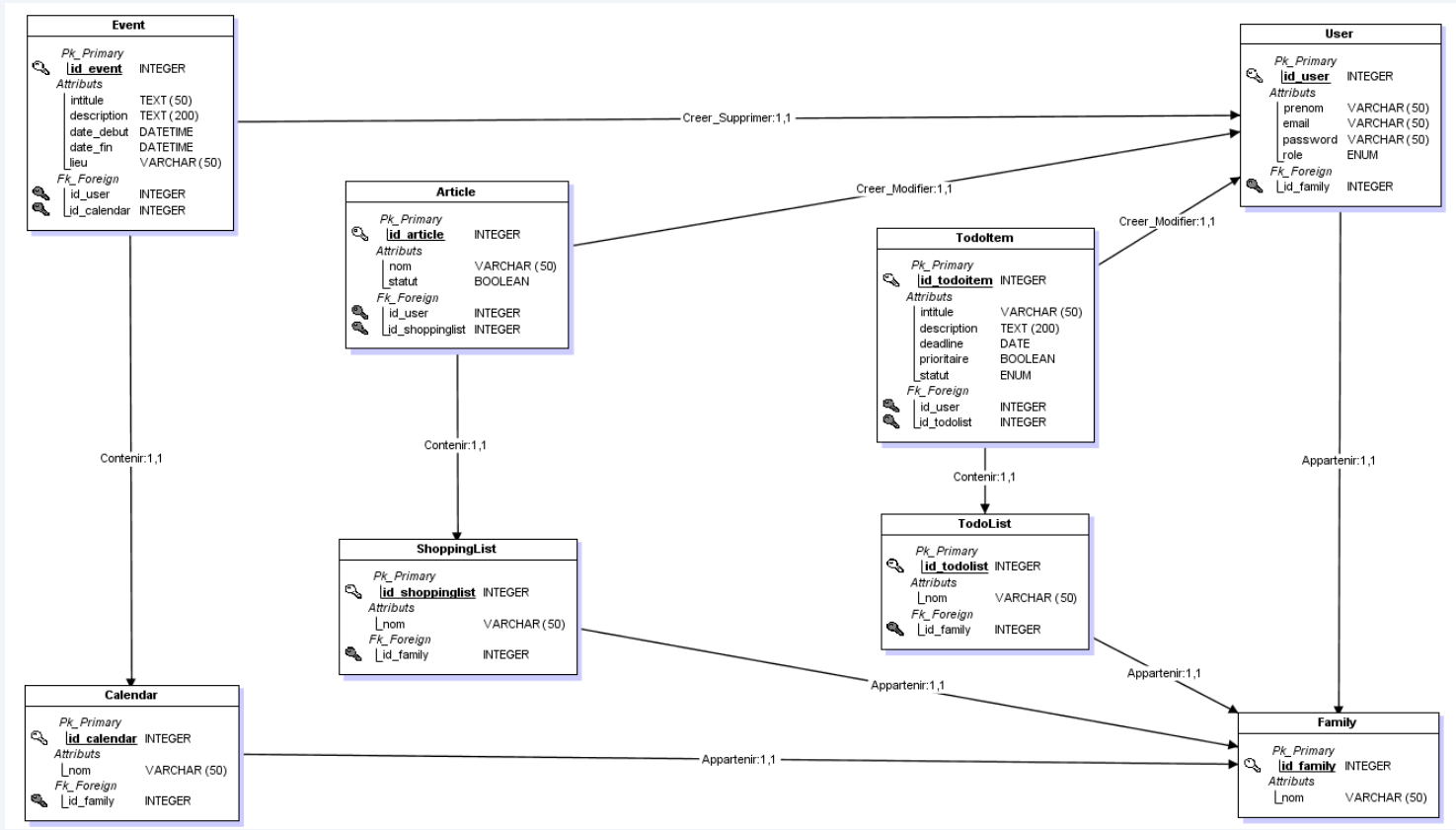
09. Annexes Techniques

Slides additionnels

Modèles MCD et MLD

Méthode Merise :

- **MCD :** Identification des 8 entités et des cardinalités (ex: Family 1,n <---> 1,1 User).
- **MLD :** Traduction en tables avec clés primaires (PK) et clés étrangères (FK)
- **Intégrité référentielle :** Suppressions en cascade (ON DELETE CASCADE)



[Modèle Logique des Données (MLD)]